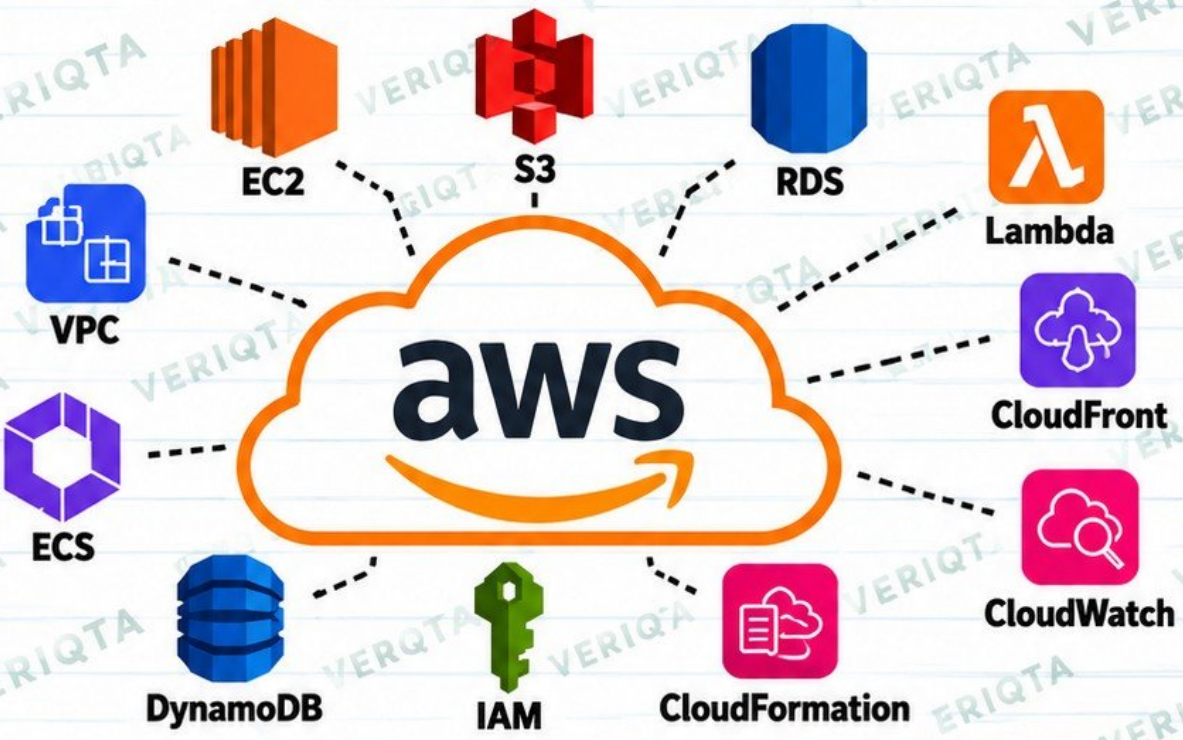
2026 Edition

10 AWS HANDS-ON PROJECTS FOR BEGINNERS

BUILD | DEPLOY | CONFIGURE | TROUBLESHOOT | MONITOR



- Real-World Projects
- Step-by-Step Guides
- Practical Examples
- Common Failures and Fixes
- Best Practices
- Clean Up Resources

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

Project 01. Secure Your AWS Environment

AWS IAM, MFA, CloudTrail, AWS Budgets

1. What You Will Build

- Understand root users, IAM users, roles, and permissions.
- Enable multifactor authentication (MFA).
- Create a least-privilege learning identity.
- Understand temporary credentials and permission boundaries.

2. Why This Matters

- Protects your AWS account.
- Prevents unauthorized access.
- Helps you follow least privilege.
- MFA stops attackers even if they get your password.
- These are the foundations of real-world cloud security.

3. Key Concepts

Root User
- Full access to everything.
- Use only for account recovery.
- Do not use for daily work.

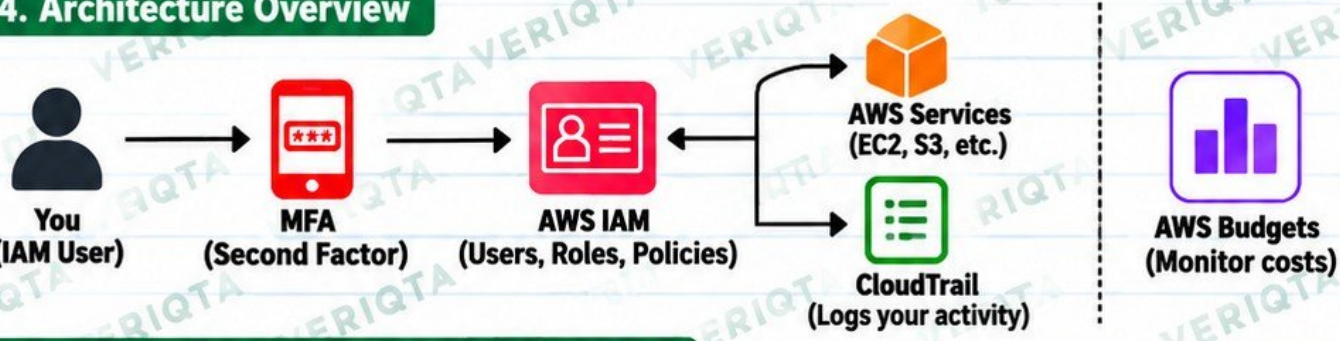
IAM User
- Individual identity for a person.
- Give only the permissions needed (least privilege).

IAM Role
- Temporary permissions.
- Used by services such as EC2, Lambda, or by other accounts.
- More secure than long-term credentials.

MFA
- Adds a second verification step.
- Stops attackers even if they have your password.

Policies
- Define what a user or role can do.
- Use least privilege.
- Avoid giving unnecessary permissions.

4. Architecture Overview



5. Steps to Build a Secure AWS Starting Point

1. Sign in to the AWS Management Console using your root account.
2. Enable multifactor authentication (MFA) for the root user.
3. Create a new IAM user for daily use.
4. Attach a least-privilege policy to the IAM user (for example, read-only or specific service access).
5. Test the IAM user by signing in with the new credentials.
6. Explore temporary credentials and IAM roles.
7. Check your account activity in CloudTrail.
8. Set up an AWS Budgets alert to monitor costs.

5. Best Practices

- Always use MFA.
- Follow least privilege.
- Use IAM roles when possible.
- Regularly review permissions.
- Monitor activity with CloudTrail.
- Set up a budget to avoid unexpected charges.

7. Next Page

Page 02: Test Permissions and Protect Your Account

- Verify allowed and denied actions.
- Inspect CloudTrail activity.
- Configure a cost budget and alert.
- Practice diagnosing AccessDenied.
- Remove unnecessary permissions and credentials.

@veriqta Instagram
@veriqta GitHub
@veriqta LinkedIn
@veriqta Telegram
@veriqta X
@veriqta Discord
VERIQTA

Project 01. Secure Your AWS Environment

AWS IAM, MFA, CloudTrail, AWS Budgets

Test Permissions and Protect Your Account

1. Test Allowed and Denied Actions

Use the AWS CLI to test what your IAM user can and cannot do. This helps you confirm your permissions are working as expected.

```
# Check your current identity
aws sts get-caller-identity
# List S3 buckets (allowed if you have permission)
aws s3 ls
# Try to access a service you don't have permission for
aws ec2 describe-instances
```

Example AccessDenied Output

An error occurred (AccessDenied) when calling the DescribeInstances operation: User is not authorized to perform: ec2:DescribeInstances

2. Inspect CloudTrail Activity

CloudTrail records API calls made in your AWS account. You can use it to see what actions were allowed or denied, and when they happened.

CloudTrail

Lookup events

Event time	Event name	Event source	Error code
2026-10-20 12:15	ListBuckets	s3.amazonaws.com	-
2026-10-20 12:18	DescribeInstances	ec2.amazonaws.com	AccessDenied
2026-10-20 12:20	GetCallerIdentity	sts.amazonaws.com	-

What to Look For

- Which user or role made the request.
- What action was performed.
- Whether the action was allowed or denied.
- The time and source (console, CLI, or API).

3. Configure a Cost Budget and Alert

An AWS Budget helps you monitor your spending and get an alert before unexpected charges.

1. Open the AWS Budgets console.
2. Click Create budget.
3. Choose Cost budget.
4. Set your monthly limit (for example, $5 or $10).
5. Add an email alert (for example, 80% and 100%).
6. Review and create the budget.



4. Diagnose AccessDenied Errors

AccessDenied means your IAM user or role does not have the required permission. Use this approach to troubleshoot.

1. Read the error message carefully.
2. Identify the service and action (for example, ec2:DescribeInstances).
3. Check the IAM policy attached to your user or role.
4. Confirm whether the action is explicitly allowed or denied.
5. Update the policy with the correct permission (least privilege).
6. Test the action again to verify it works.

Common Causes of AccessDenied

- Permission not attached to the user or role.
- Policy does not include the required action.
- Resource-level restriction (for example, specific bucket).
- Explicit Deny in a policy.
- Using the wrong region or resource ID.

Tip Always use the least privilege policy. Only allow the permissions needed for the task.

5. Remove Unnecessary Permissions and Credentials

After testing, clean up to keep your account secure.

- Remove permissions that are no longer needed.
- Detach unused policies from IAM users or roles.
- Delete unused access keys.
- Ensure MFA is enabled on important accounts.
- Review account activity in CloudTrail regularly.

Security Reminder

Never keep long-term credentials that you do not use. Use temporary credentials or IAM roles when possible. Regularly review permissions and CloudTrail activity to detect unexpected actions.

Project 02. Host a Static Website on AWS

Amazon S3, CloudFront, Origin Access Control

1. What You Will Build

- Create an S3 bucket and upload a simple HTML website.
- Keep the bucket private.
- Configure CloudFront with Origin Access Control.
- Understand how a browser retrieves website files.

2. What You Will Learn

- How Amazon S3 stores website files.
- Why we use CloudFront for faster and more secure delivery.
- What Origin Access Control (OAC) is.
- How to keep the S3 bucket private.
- How a browser gets and displays the files.

3. Architecture Overview



1. The browser requests your website using the CloudFront domain name.
2. CloudFront securely retrieves the files from your S3 bucket using Origin Access Control.
3. CloudFront caches the files at edge locations and returns them to the browser.

Why Use CloudFront and OAC?

- Faster delivery using global edge locations.
- Keeps your S3 bucket private.
- Prevents direct public access to your bucket.
- More secure and recommended by AWS.

4. Create an S3 Bucket and Upload a Website

1. Open the AWS S3 console.
2. Create a bucket with a unique name.
3. Keep Block Public Access enabled (bucket remains private).
4. Upload a simple HTML file.
5. Confirm the file is uploaded.

Example index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Hello from VERIQTA</title>
</head>
<body>
    <h1>Welcome to VERIQTA on AWS!</h1>
    <p>This is a simple static website hosted
     with S3 and CloudFront.</p>
</body>
</html>
```

5. Configure CloudFront with Origin Access Control

1. Open the CloudFront console.
2. Create a new distribution.
3. Select your S3 bucket as the origin.
4. Choose Origin Access Control (OAC) and keep the bucket private.
5. Use default viewer settings (HTTPS).
6. Create the distribution and wait for it to deploy.
7. Copy the CloudFront domain name.

6. Test Your Website

- Open the CloudFront domain name in a browser.
- You should see your website content.
- Try refreshing the page multiple times.
- Your S3 bucket should remain private.

⚠ Important Security Check

- Do not enable public access on the S3 bucket.
- Do not use the S3 bucket website endpoint.
- Always use CloudFront with Origin Access Control.
- Track your resources to avoid unexpected charges.

Project 02. Host a Static Website on AWS

Amazon S3, CloudFront, Origin Access Control

Troubleshoot Website Delivery

1. Test the CloudFront Distribution URL

1. Open your CloudFront distribution domain name in a browser.
2. You should see your website (for example, index.html).
3. Try refreshing the page multiple times.
4. Test another file (for example, about.html).
5. Verify that the website loads from the CloudFront URL and not directly from the S3 bucket.

How the Request Works



2. Investigate 403 AccessDenied

A 403 AccessDenied error usually means CloudFront cannot access the file from S3 or the file is missing.

Common causes

- ✗ Origin Access Control (OAC) not configured correctly.
- ✗ S3 bucket is not set to block public access.
- ✗ Bucket policy does not allow CloudFront access.
- ✗ File does not exist or wrong file path.
- ✗ Trying to access the S3 bucket directly.
- ✗ CloudFront distribution is still deploying.

Example 403 Error



How to Fix

- ✓ Confirm OAC is attached to the distribution.
- ✓ Check the S3 bucket policy for CloudFront.
- ✓ Verify the file exists in the bucket.
- ✓ Use the correct file path (for example, index.html).
- ✓ Wait for the distribution to deploy fully.

3. Investigate Missing Files (404)

A 404 error means the file does not exist at the requested path.

Debug steps

1. Check the file name and path in S3.
2. Make sure the file is uploaded to the correct folder.
3. Verify the default root object (index.html) is set.
4. Check for case sensitivity (Index.html ≠ index.html).
5. Inspect CloudFront error logs if enabled.

4. Understand Caching and Invalidations

- CloudFront caches files at global edge locations.
- After you update a file, old content may still load for a short time.
- To serve the new version immediately, create an invalidation.

Create an invalidation (AWS CLI)

```
aws cloudfront create-invalidation \
  --distribution-id E123ABC456 \\
  --paths "/index.html" "/about.html"\
```

5. Verify Direct S3 Access is Blocked

Your S3 bucket should remain private. Users shouldt be able to access files directly using the S3 URL.

Test direct access

1. Open the S3 object URL in a browser.
2. You should see an AccessDenied error.
3. If the file is accessible, block public access and check your bucket policy.

Expected Result



Why This Matters

- ✓ Keeps your files private.
- ✓ Ensures traffic goes through CloudFront.
- ✓ Protects against unintended public access.

6. Remove the Distribution and Bucket After Testing

1. Delete the CloudFront distribution (wait for deletion to complete).
2. Delete the S3 bucket and all objects inside it.
3. Confirm both resources are removed.
4. Check AWS Budgets to ensure no unexpected charges.

⚠ Important

- ✗ Deleting a CloudFront distribution can take time.
- ✗ Delete all objects in the S3 bucket before deleting the bucket.
- ✗ Ensure there are no other resources attached.
- ✗ Always clean up to avoid unnecessary charges.

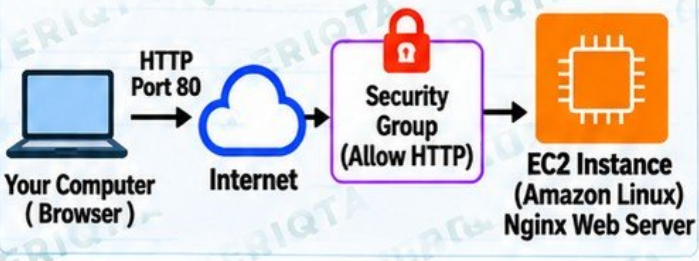
Project 03. Deploy a Linux Web Server

Amazon EC2, Linux, Security Groups, Systems Manager

1. What You Will Build

- Select an appropriate Amazon Linux image.
- Launch a small EC2 instance.
- Configure inbound HTTP access.
- Connect using Systems Manager where supported.
- Install and start Nginx.

2. Architecture Overview



3. Choose an Amazon Linux Image

Amazon Linux
2023 (Recommended)

- Free tier eligible (t2.micro / t3.micro).
- Includes modern packages.
- Good for learning and production.
- Keep the default settings for a beginner setup.

4. Launch an EC2 Instance

1. Open the EC2 console.
2. Click Launch instance.
3. Choose Amazon Linux 2023.
4. Select t2.micro or t3.micro (free tier eligible).
5. Create a key pair (optional if using Systems Manager).

5. Configure a Security Group

Type	Protocol	Port	Source	Purpose
HTTP	TCP	80	0.0.0.0/0	Allow web traffic from anywhere (for testing)
SSH	TCP	22	Your IP (optional)	Only if you plan to use SSH

Security Tips

- Allow only required ports.
- Use your IP for SSH if needed.
- Remove unused rules later.
- Do not use 0.0.0.0/0 for SSH.
- Use a separate security group for each project.

6. Connect Using Systems Manager

Systems Manager Session Manager allows you to connect to your instance without an SSH key (if the IAM role is configured).

1. Make sure the instance has the AmazonSSMManagedInstanceCore role.
2. Open the EC2 console.
3. Select your instance.
4. Click Connect → Session Manager.
5. Start a session in your browser.

Benefits

- No SSH key required.
- More secure.
- Works over HTTPS.
- Useful for beginner and production environments.



7. Install and Start Nginx

```
# Update the system
sudo dnf update -y
# Install Nginx
sudo dnf install nginx -y
# Start Nginx
sudo systemctl start nginx
sudo systemctl enable nginx
# Check status
sudo systemctl status nginx
```

8. Verify Your Website

Open your EC2 public IP in a browser:

`http://<your-ec2-public-ip>`



9. Important Notes

- Your instance needs a public IP to access it from the internet.
- Use a small instance type (t2.micro or t3.micro).
- Stop or terminate the instance after testing to avoid charges.
- For production, do not allow HTTP from anywhere. Use a load balancer and additional security.

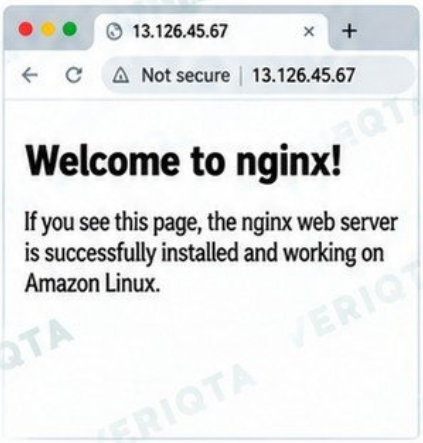
Project 03. Deploy a Linux Web Server

Amazon EC2, Linux, Security Groups, Systems Manager

Diagnose a Failed Web Server

1. Test the Instance's Public Endpoint

Open the public IP or DNS name of your EC2 instance in a browser to check if the website loads.



If it works

- You see the nginx welcome page.
- Port 80 is open.
- The web server is running.

If it fails

- Page does not load.
- Connection timeout.
- 403 or 502 error.
- Check the next steps.

2. Inspect systemctl status nginx

Connect to your instance (using Systems Manager or SSH) and check the status of the nginx service.

```
ec2-user@ip-10-0-1-10:~$ systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service;
     Active: active (running) since Mon 2026-10-20 12:34:56
       Docs: man:nginx(8)
   Main PID: 1234 (nginx)
      Tasks: 3 (limit: 1137)
     Memory: 5.2M
     CGroup: /system.slice/nginx.service
             ├─ 1234 nginx: master process
             ├─ 1235 nginx: worker process
             └─ 1236 nginx: worker process
```

What to check

- Service is active (running).
- No errors in the status output.
- Worker processes are started.

3. Investigate Connection Timeouts

If the website does not load, check where the problem is.

1. Try to access http://<public-ip> from a browser.
2. Use curl to test connectivity.
3. Check if port 80 is open.
4. Verify the instance is running.
5. Look for network or security group issues.

```
[user@local ~]$ curl -I http://13.126.45.67
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: text/html
Content-Length: 615

[user@local ~]$ telnet 13.126.45.67 80
Trying 13.126.45.67...
Connected to 13.126.45.67.
```

4. Distinguish the Cause

Use this guide to identify whether the problem is a security group issue or an application issue.

Security Group Failure

- Connection timeout.
- Cannot reach port 80.
- Security group does not allow HTTP (port 80) from your IP or anywhere (0.0.0.0/0 for testing).
- EC2 instance may be running, but traffic is blocked.

Application Failure

- Can connect to the instance, but web page does not load.
- systemctl status nginx shows inactive or failed.
- Nginx is not installed or not running.
- Check logs with: `sudo journalctl -u nginx -xe`

5. Common Errors and Fixes

Connection Timeout

- Security group does not allow port 80.
- Wrong public IP or instance stopped.
- Fix: Edit security group and add HTTP (TCP 80) from 0.0.0.0/0.

403 AccessDenied

- You are accessing the wrong service (for example, S3 instead of EC2).
- Bucket or resource policy issue.
- Fix: Ensure you are using the EC2 public IP or correct domain.

Nginx Service Not Running

- Nginx is not installed or stopped.
- Check the status and install if needed.
- Fix: sudo systemctl start nginx and enable it.

502 Bad Gateway

- Nginx is not running correctly.
- Port 80 is open, but the application is failing.
- Fix: Check nginx logs and configuration.

6. Terminate the Instance After Testing

Remove the EC2 instance to avoid unexpected charges.

1. Open the EC2 console.
2. Select your instance.
3. Click Instance state → Terminate instance.
4. Confirm termination.
5. Verify the instance state changes to Terminated.

7. Verify Cleanup

- Instance state is Terminated.
- Elastic IP (if used) is released.
- Security group is no longer needed and can be deleted.
- No unexpected charges in AWS Billing (check after a few hours).

Important

- Terminate the instance when you are finished testing.
- Remove unused security groups.
- Check AWS Billing to ensure there are no unexpected charges.
- Do not leave test resources running.

Project 04. Build Your First VPC Network

Amazon VPC, Subnets, Route Tables, Internet Gateway

Create a Custom Cloud Network

1. Plan a VPC CIDR Block

A VPC is a private network in AWS. You choose an IP address range (CIDR block) for your VPC and then create subnets inside it.

Example CIDR block

10.0.0.0/16

- Provides 65,536 IP addresses.
- You can divide it into smaller subnets.
- Common for learning and small projects.

Subnet planning (example)

- Public Subnet 1: 10.0.1.0/24
- Public Subnet 2: 10.0.2.0/24
- Private Subnet 1: 10.0.3.0/24
- Private Subnet 2: 10.0.4.0/24

2. Create Public and Private Subnets

1. Create a VPC with the CIDR block (for example, 10.0.0.0/16).
2. Create at least two public subnets (in different Availability Zones).
3. Create at least two private subnets.
4. Each subnet should have a unique CIDR block.
5. Choose the Availability Zone for each subnet.

3. Attach an Internet Gateway (IGW)

1. Create an Internet Gateway.
2. Attach the IGW to your VPC.
3. The IGW allows resources in public subnets to access the internet.
4. Private subnets do not route traffic directly to the internet.

Internet Gateway



Internet Gateway (IGW)

Connects your VPC to the internet.

4. Configure Route Tables

1. Create a route table for public subnets.
2. Add a route to 0.0.0.0/0 pointing to the Internet Gateway.
3. Associate the public subnets with this route table.
4. Create a separate route table for private subnets.
5. Do not add a route to the Internet Gateway for private subnets (keep them private).

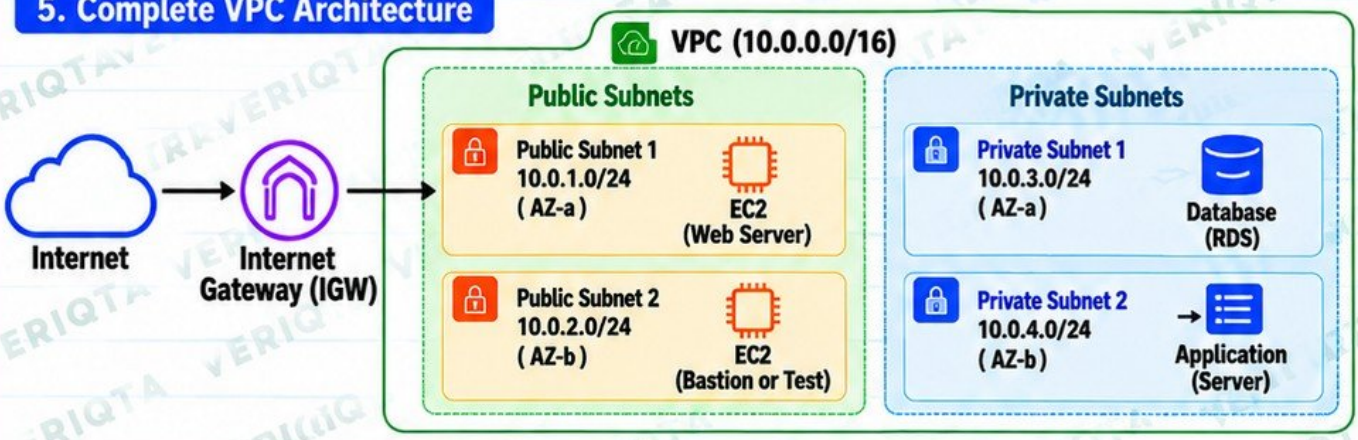
Public Route Table (example)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxxx

Private Route Table (example)

Destination	Target
10.0.0.0/16	local

5. Complete VPC Architecture



6. Why a Subnet is Public or Private?

Public Subnet

- Has a route to the Internet Gateway (0.0.0.0/0 → IGW).
- Resources can access the internet.
- Used for web servers, load balancers, or bastion hosts.

Private Subnet

- Does not have a route to the Internet Gateway.
- Resources cannot be accessed directly from the internet.
- Used for databases, backend services, and internal applications.

Key Points

- The Internet Gateway is attached to the VPC, not to a subnet.
- Only subnets with a route to the IGW are public.
- Private subnets can access the internet using a NAT Gateway (learn this in a later project).
- Use multiple Availability Zones for better reliability.

Project 04. Build Your First VPC Network

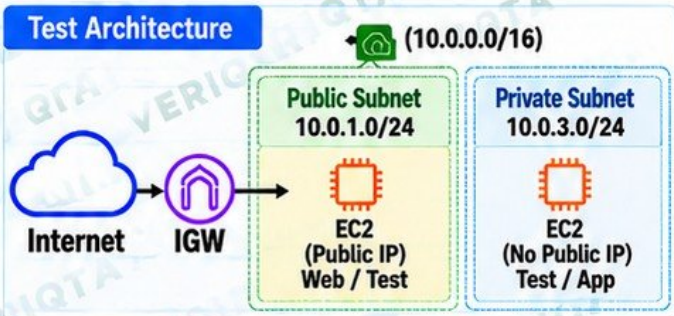
Amazon VPC, Subnets, Route Tables, Internet Gateway

Investigate a Network Connectivity Failure

1. Launch a Test Instance

1. Launch a small EC2 instance in a public subnet.
2. Launch another small EC2 instance in a private subnet.
3. Use Amazon Linux and a t2.micro or t3.micro.
4. Connect to the public instance using its public IP or Systems Manager.
5. For the private instance, use Systems Manager (no public IP).

Test Architecture



2. Inspect Routes and Security Groups

Check Route Table (Public Subnet)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxxxx

Public subnets must have a route to the Internet Gateway (0.0.0.0/0 → IGW).

Check Route Table (Private Subnet)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-xxxxxxxxx (optional)

Private subnets do not have a route to IGW. Use a NAT Gateway if you need internet access.

Security Group Example (Web Server)

Type	Protocol	Port	Source	Purpose
HTTP	TCP	80	0.0.0.0/0	Allow web traffic
SSH	TCP	22	Your IP	Allow SSH (from your IP)

3. Test Network Connectivity

From the public instance

```
[ec2-user@ip-10-0-1-10 ~]$ ping -c 4 8.8.8.8
[ec2-user@ip-10-0-1-10 ~]$ curl -I https://www.amazon.com
```

✓ Should see successful responses (internet access).

From the private instance

```
[ec2-user@ip-10-0-3-10 ~]$ ping -c 4 8.8.8.8
[ec2-user@ip-10-0-3-10 ~]$ curl -I https://www.amazon.com
```

✗ If there is no NAT Gateway, these commands will fail (connection timeout).

Common Errors and Causes

Error / Symptom	Likely Cause
Connection timeout	No route to the internet (NAT Gateway missing).
Cannot reach website	Wrong route table or blocked security group.
Port 80 not reachable	Security group does not allow HTTP (port 80).
SSH not working	Wrong source IP or missing SSH rule.
Works in public but not private	Private subnet has no NAT Gateway or internet route.

4. Private-Subnet Outbound Access Options

NAT Gateway (Recommended for production)

- ✓ Allows private subnets to access the internet.
- ✓ Highly available in an AZ.
- ✓ Incurs hourly and data charges.
- ✓ Use only when internet access is required.

NAT Instance (Lower cost option)

- ✓ An EC2 instance acting as a NAT.
- ✓ Cheaper than NAT Gateway.
- ✓ Requires maintenance and patching.
- ✓ Use for learning or non-production.

VPC Endpoints (Best for AWS services)

- ✓ Allows private subnets to access AWS services (S3, DynamoDB, etc.) without internet.
- ✓ More secure and no internet charges.
- ✓ Does not provide access to the general internet.

5. Diagnose and Fix the Issue

1. Check the route table for the subnet.
2. Verify the security group rules.
3. Ensure a NAT Gateway or VPC endpoint is configured if needed.
4. Test connectivity again.
5. Confirm the application or service is working.

6. Avoid Unnecessary NAT Gateway Charges

- ✓ Only create a NAT Gateway when private subnets need internet access.
- ✓ Use VPC endpoints for AWS services instead of NAT.
- ✓ Delete the NAT Gateway after testing if not needed.
- ✓ Monitor costs using AWS Budgets.
- ✓ Consider a NAT instance for non-production learning.

⚠ Clean Up After Testing

1. Terminate test EC2 instances.
2. Delete the NAT Gateway (if created).
3. Remove any VPC endpoints (if not needed).
4. Delete the VPC and all resources.
5. Confirm in AWS Billing that resources are deleted.

Project 05. Build a Serverless API

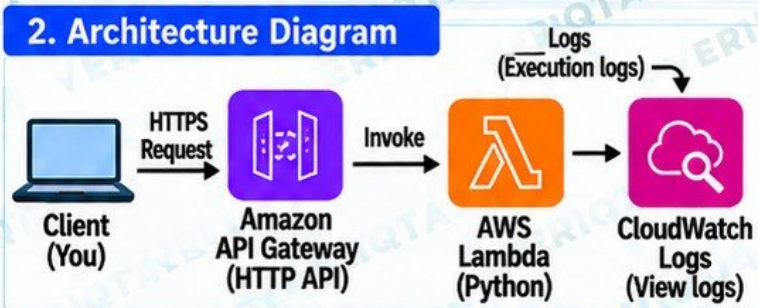
AWS Lambda, API Gateway, CloudWatch Logs

Deploy an API Without Managing Servers

1. Project Overview

You will build a simple serverless API using AWS Lambda and API Gateway. The API will return a JSON message when you send a request. You will also view the logs in CloudWatch.

2. Architecture Diagram



3. Write a Simple Python Lambda Function

Python (lambda_function.py)

```python
import json

def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({
            "message": "Hello from VERIQTA API!",
            "method": event.get("requestContext", {}).get("http",
                                {}).get("method"),
            "time": context.aws_request_id
        })
    }
```

4. Create the Lambda Function

1. Open the AWS Lambda console.
2. Click Create function.
3. Choose Author from scratch.
4. Use a name (for example, veriqta-api).
5. Select Python 3.12 (or the latest).
6. Paste your code and click Deploy.
7. Keep the default execution role (creates a role with basic permissions).

5. Create an API Gateway HTTP API

1. Open the API Gateway console.
2. Click Create API and choose HTTP API.
3. Add an integration with your Lambda function.
4. Create a route (for example, GET /).
5. Deploy the API (create a new stage, e.g. $default).
6. Copy the API endpoint URL.

API Gateway Settings (Example)

API type	HTTP API
Route	GET /
Integration	Lambda function
Stage	$default
Invoke URL	https://abc123.execute-api.region.amazonaws.com

6. Test the API

1. Open the Invoke URL in a browser or use curl.
2. You should see a JSON response from your function.
3. Try changing the route or method and test again.

Test with curl

```
curl https://abc123.execute-api.region.amazonaws.com/
```

Example Response

https://abc123.execute-api.region.amazonaws.com/

```
{
  "message": "Hello from VERIQTA API!",
  "method": "GET",
  "time": "a1b2c3d4-e5f6-7890-abcd-1234567890ef"
}
```

7. View Execution Logs in CloudWatch

1. Open the CloudWatch console.
2. Go to Log groups.
3. Find the log group for your Lambda function (/aws/lambda/your-function-name).
4. Open the latest log stream to view request logs.

8. Understand the Request and Response Flow

1. The client sends an HTTPS request to the API Gateway URL.
2. API Gateway routes the request to your Lambda function.
3. Lambda runs your code and returns a response.
4. API Gateway sends the response back to the client.
5. Execution details are written to CloudWatch Logs.

Project 05. Build a Serverless API

AWS Lambda, API Gateway, CloudWatch Logs

Debug a Failing API

1. Test the Endpoint with curl

1. Use curl to call your API endpoint.
2. Check the response and status code.
3. Try different routes or methods (GET, POST).
4. Compare the response with what you expect.

```
$ curl -i https://abc123.execute-api.region.amazonaws.com/
HTTP/2 200
content-type: application/json
{
  "message": "Hello from VERIQTA API!"
}
```

2. Inspect HTTP Status Codes

Status codes tell you what happened with your request.

Status Code	Meaning	Common Cause	What to Do
200	Success	Request worked	Check response body
400	Bad Request	Invalid request data	Check request format and headers
403	Forbidden	Permission denied	Check IAM roles and resource policy
404	Not Found	Wrong route or stage	Verify URL, route and HTTP method
500	Internal Error	Lambda error	Check CloudWatch Logs for details

3. Investigate Lambda Exceptions

1. Open the AWS Lambda console.
2. Go to the function and check the latest invocations.
3. Look for error messages and stack traces.
4. Fix code errors, missing environment variables, or incorrect permissions.

Example Lambda Error (from logs)

```
[ERROR] NameError: name 'name' is not defined
Traceback (most recent call last):
  File "/var/task/lambda_function.py", line 5, in lambda_handler
    return {"message": f"Hello {name}"}
NameError: name 'name' is not defined
```

4. Use CloudWatch Logs to Find Evidence

1. Open the CloudWatch console.
2. Go to Log groups.
3. Find the log group for your Lambda function (/aws/lambda/your-function-name).
4. Open the latest log stream.
5. Check logs for errors, timeouts or permission issues.

CloudWatch Logs (Example)

```
START RequestId: a1b2c3d4-1234-5678-90ab-cdef12345678
EVENT {"httpMethod": "GET", "path": "/"}
[ERROR] AccessDeniedException: User is not authorized
to perform: logs:CreateLogStream on resource:
arn:aws:logs:region:123456789012:log-group:/aws/lambda/my-api
END RequestId: a1b2c3d4-1234-5678-90ab-cdef12345678
REPORT RequestId: a1b2c3d4-1234-5678-90ab-cdef12345678
Duration: 120.45 ms  Billed Duration: 121 ms  Memory Size: 128 MB
Max Memory Used: 75 MB
```

5. Common API Problems and Fixes

Problem	Example Error	Likely Cause	How to Fix
403 Forbidden	{"message":"Forbidden"}	Missing invoke permission	Add API Gateway invoke permission to Lambda
404 Not Found	{"message":"Not Found"}	Wrong route or stage	Check API URL, route and method
500 Internal Error	Internal server error	Lambda code error	Check CloudWatch Logs and fix the code
Timeout	504 Gateway Timeout	Function running too long	Optimize code or increase timeout
CORS Error	CORS policy error	Missing CORS headers	Enable CORS in API Gateway

6. Correct the Problem and Verify

1. Make the required code or permission changes.
2. Deploy the Lambda function (if code changed).
3. Test the endpoint again with curl.
4. Confirm you get the expected response and status code 200.

Successful Response (After Fix)

```
$ curl -i https://abc123.execute-api.region.amazonaws.com/
HTTP/2 200
content-type: application/json
{
   "message": "Hello from VERIQTA API!",
}  "status": "success"
```

7. Key Takeaways

- Use curl to test and check status codes.
- CloudWatch Logs show the real error messages.
- Most issues are permissions, wrong routes or code errors.
- Fix the problem, deploy and test again.
- Always verify you get the correct response.

Project 06. Create a Cloud Database

Amazon DynamoDB, IAM, AWS CLI

Build a Task Tracking Database

1. Create a DynamoDB Table

1. Open the DynamoDB console.
2. Click Create table.
3. Enter a table name (for example, veriqta-tasks).
4. Choose a partition key (for example, taskId, String).
5. Keep default settings and create the table.

DynamoDB - Create table

Table name: veriqta-tasks

Partition key: taskId | String

▸ Additional settings

2. Understand the Table Design

- A partition key (also called a primary key) uniquely identifies each item.
- DynamoDB uses the partition key to store and retrieve data at high speed.
- For this project, we use taskId as the partition key.
- You can add other attributes such as title, status and createdAt.
- Applications use the partition key to read, update and delete specific items.

Table Structure (Example)

Attribute	Type	Description
taskId	String (Partition Key)	Unique ID for each task
title	String	Task title
status	String	Task status (e.g. todo, in-progress, done)
createdAt	String	Date and time the task was created

3. Add Sample Task Records (AWS CLI)

```
aws dynamodb put-item \
--table-name veriqta-tasks \
--item '{
  "taskId": {"S": "1"},
  "title": {"S": "Learn AWS"},
  "status": {"S": "todo"},
  "createdAt": {"S":
     "2026-10-20T10:00:00Z"}
}'
```

Add another item:

```
aws dynamodb put-item \
--table-name veriqta-tasks \
--item '{
  "taskId": {"S": "2"},
  "title": {"S": "Build a Project"},
  "status": {"S": "in-progress"},
  "createdAt": {"S":
     "2026-10-20T11:00:00Z"}
}'
```

4. Read, Update and Delete Records

Read an item

```
aws dynamodb get-item \
  --table-name veriqta-tasks \
  --key '{"taskId": {"S": "1"}}'
```

Update an item

```
aws dynamodb update-item \
  --table-name veriqta-tasks \
  --key '{"taskId": {"S": "1"}}' \
  --update-expression "SET #s = :s" \
  --expression-attribute-names
   {"#s": "status"}' \
  --expression-attribute-values
    '{":s": {"S": "done"}}'
```

Delete an item

```
aws dynamodb delete-item \
  --table-name veriqta-tasks \
  --key '{"taskId": {"S": "1"}}'
```

5. Verify the Data in the Console

Items (2) — Actions ▾ — Create item

taskId	title	status	createdAt
1	Learn AWS	done	2026-10-20T10:00:00Z
2	Build a Project	in-progress	2026-10-20T11:00:00Z

6. Create an IAM User for CLI Access

1. Open the IAM console.
2. Create a new user (for example, veriqta-cli).
3. Attach the AmazonDynamoDBFullAccess policy (or a more restricted policy for real use).
4. Create an access key.
5. Configure the AWS CLI with the credentials.

7. Configure the AWS CLI

1. Install the AWS CLI (if not installed).
2. Run aws configure.
3. Enter your access key, secret key, region and output format.

```
aws configure
AWS Access Key ID [None]: YOUR_ACCESS_KEY
AWS Secret Access Key [None]: YOUR_SECRET_KEY
Default region name [None]: us-east-1
Default output format [None]: json
```

8. How Applications Use the Key

- The partition key (taskId) uniquely identifies each item.
- Applications use the key to read a specific task.
- The same key is used to update or delete the task.
- Choose a key that is unique and meaningful.
- In real applications, you can use a composite key (partition key + sort key) for more advanced queries.

Key Points

- DynamoDB is a NoSQL database.
- Data is stored as items with attributes.
- The partition key is required.
- Use the AWS CLI or console to manage data.
- Applications use the key to read, update and delete specific items.
- Design your key properly for your access patterns.

Amazon DynamoDB, IAM, AWS CLI

Troubleshoot Database Operations

1. Test Queries Using Known Keys

1. Use the partition key (taskId) to query a specific item.
2. Verify the item exists and check all attributes.
3. Try querying with a key that does not exist.
4. Confirm the response and understand the result.

Query a specific item

```
aws dynamodb get-item \
--table-name veriqta-tasks \
--key '{"taskId": {"S": "1"}}'
```

Successful response (item found)

```
{
  "Item": {
    "taskId": {"S": "1"},
    "title": {"S": "Learn AWS"},
    "status": {"S": "done"}
  }
}
```

Item not found (valid response)

```
{
  "Item": null
}
```

2. Understand Scan vs Query

Feature	Query	Scan
Purpose	Find items using a key (condition)	Read all items in the table
Uses partition key	Yes (required)	No
Performance	Fast and efficient	Slower (reads entire table)
Best practice	Use for specific items	Use only for small tables or troubleshooting
Cost	Lower (reads fewer items)	Higher (reads all items)

Query example (recommended)

```
aws dynamodb query \
 --table-name veriqta-tasks \
 --key-condition-expression "taskId = :id" \
 --expression-attribute-values '{":id": {"S": "1"}}'
```

Scan example (reads entire table)

```
aws dynamodb scan \
 --table-name veriqta-tasks
```

3. Investigate Missing Items

1. Confirm you are using the correct table name and region.
2. Check the key value (taskId) is correct.
3. Verify the item was actually created.
4. Look for typographical errors in the key.

Common reasons for missing items

- ✗ Using the wrong table name or region.
- ✗ Incorrect partition key value.
- ✗ Item was never created.
- ✗ Eventual consistency delay (item may not appear immediately).
- ✗ Using Scan with filters instead of Query with the key.

4. Investigate Denied Permissions

1. Check the error message for AccessDeniedException.
2. Verify your IAM user or role has the correct DynamoDB permissions.
3. Ensure the policy allows the specific action (GetItem, PutItem, Query, etc.).
4. Check that you are using the correct AWS credentials and region.

Example error (Access denied)

```
{
  "__type": "com.amazon.coral.service#AccessDeniedException",
  "message": "User is not authorized to perform:
  dynamodb:GetItem on resource: arn:aws:dynamodb:..."
}
```

5. Avoid Overwriting Data Accidentally

- ✓ Use a unique key value for each item.
- ✓ When updating items, use UpdateItem instead of PutItem if you only want to change specific attributes.
- ✓ Understand that PutItem can overwrite an existing item with the same key.
- ✓ Optionally use a condition expression to prevent accidental overwrites.

Update only the status attribute

```
aws dynamodb update-item \
 --table-name veriqta-tasks \
 --key '{"taskId": {"S": "1"}}' \
 --update-expression "SET #s = :status" \
 --expression-attribute-names '{"#s": "status"}' \
 --expression-attribute-values '{":status": {"S": "in-progress"}}'
```

6. Verify Records Before Deleting the Table

1. List all items or run a scan to confirm your data.
2. Check that you have the information you need.
3. If required, export or save the data before deletion.
4. Then delete the table.

List all items (verify data)

```
aws dynamodb scan --table-name veriqta-tasks
```

Delete the table

```
aws dynamodb delete-table --table-name veriqta-tasks
```

Key Points

- ✓ Use Query with the partition key for efficient lookups.
- ✓ Scan reads the entire table and can be slow and expensive.
- ✓ Check IAM permissions when you see AccessDeniedException.
- ✓ Use UpdateItem to avoid overwriting data.
- ✓ Always verify your records before deleting the table.

Project 07. Automate Infrastructure Deployment

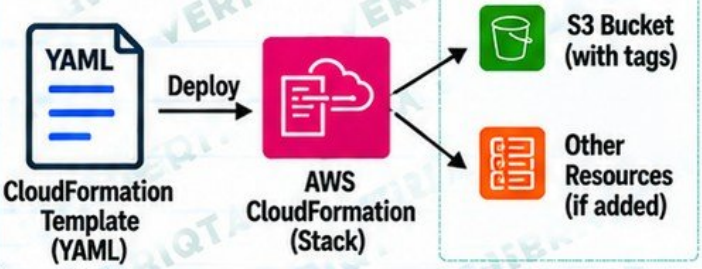
AWS CloudFormation, YAML, CloudFormation Events

Deploy Infrastructure Using Code

1. What is Infrastructure as Code (IaC)?

1. IaC lets you define and manage cloud resources using code instead of manual clicks.
2. You can create, update and delete infrastructure in a repeatable and consistent way.
3. AWS CloudFormation is an IaC service that uses YAML or JSON templates.
4. IaC reduces human error and makes deployments faster, safer and easier to reproduce.

2. Architecture Overview



3. Write a Simple CloudFormation YAML Template

template.yaml

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: Create a simple S3 bucket with tags
Resources:
  VeriqtaBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: veriqta-demo-bucket-12345
      Tags:
        - Key: Project
          Value: Veriqta
        - Key: Environment
          Value: Dev
Outputs:
  BucketName:
    Description: Name of the created S3 bucket
    Value: !Ref VeriqtaBucket
```

4. Deploy the Stack Using AWS CLI

1. Open your terminal and run the deployment command.
2. Replace the stack name and file name if needed.
3. Wait for the stack to complete.

Deploy the stack

```
aws cloudformation create-stack \
  --stack-name veriqta-s3-stack \
  --template-body file://template.yaml \
  --capabilities CAPABILITY_NAMED_IAM
```

Check stack status

```
aws cloudformation describe-stacks \
  --stack-name veriqta-s3-stack \
  --query "Stacks[0].StackStatus" \
  --output text
```

View stack outputs

```
aws cloudformation describe-stacks \
  --stack-name veriqta-s3-stack \
  --query "Stacks[0].Outputs" \
  --output table
```

5. Check the Created Resources

1. Open the AWS S3 console.
2. Find the bucket created by the stack.
3. Check the tags to confirm they were applied.
4. Open the CloudFormation console and view the stack resources.

CloudFormation Stack Resources (Example)

Logical ID	Physical ID	Type	Status
VeriqtaBucket	veriqta-demo-bucket-12345	AWS::S3::Bucket	✅ CREATE_COMPLETE

6. Compare Manual vs Repeatable Deployment

Manual Deployment (Console)

- ✗ Create resources one by one.
- ✗ Easy to make mistakes.
- ✗ Hard to reproduce the same setup.
- ✗ No version control.

CloudFormation (IaC)

- ✓ Define everything in a template.
- ✓ Consistent and repeatable.
- ✓ Faster to deploy and update.
- ✓ Can be stored in GitHub.
- ✓ Easy to delete the entire stack.

7. CloudFormation Events

1. Open the CloudFormation console.
2. Select your stack and click the Events tab.
3. Monitor the status of resource creation.
4. Use events to troubleshoot failures.

CloudFormation Events (Example)

Timestamp	Logical ID	Status	Status Reason
2026-10-20 12:01:10	VeriqtaBucket	CREATE_IN_PROGRESS	-
2026-10-20 12:01:25	VeriqtaBucket	✅ CREATE_COMPLETE	-
2026-10-20 12:01:26	veriqta-s3-stack	✅ CREATE_COMPLETE	-

Project 07. Automate Infrastructure Deployment

AWS CloudFormation, YAML, CloudFormation Events

Troubleshoot a Failed Infrastructure Stack

1. Validate Template Syntax

1. Check for YAML syntax errors (indentation, missing values).
2. Use the AWS CLI validate-template command.
3. Fix any errors and validate again.
4. A valid template must pass before deployment.

Validate using AWS CLI

```
aws cloudformation validate-template \
 --template-body file://template.yaml
```

Common Template Errors

- ✗ Incorrect indentation.
- ✗ Missing required properties.
- ✗ Invalid resource type or property name.
- ✗ Using the wrong data type (string, list, etc.).
- ✗ Missing references or parameters.

2. Inspect CloudFormation Stack Events

1. Open the CloudFormation console and select your stack.
2. Click the Events tab to see the deployment progress.
3. Look for failed resource creation and error messages.
4. Read the status reason to understand what went wrong.

Stack Events (Example)

Timestamp	Logical ID	Status	Status Reason
2026-10-20 12:01:10	VeriqtaBucket	CREATE_IN_PROGRESS	-
2026-10-20 12:02:15	VeriqtaBucket	CREATE_FAILED	Bucket name already exists
2026-10-20 12:02:16	VeriqtaS3Stack	ROLLBACK_IN_PROGRESS	The following resource(s) failed to create: [VeriqtaBucket]
2026-10-20 12:03:10	VeriqtaS3Stack	ROLLBACK_COMPLETE	-

3. Investigate Deployment and Rollback Failures

1. Identify which resource failed and why.
2. Read the status reason and error message.
3. Common causes: permissions, invalid configuration, resource already exists, or limits.
4. Check related services (S3, IAM, etc.) for more details.

Example Error from Events

```
ResourceStatus: CREATE_FAILED
ResourceType: AWS::S3::Bucket
LogicalResourceId: VeriqtaBucket
StatusReason: Bucket name 'veriqta-demo-bucket-12345'
already exists. Choose a different name.
```

4. Understand Change Sets and Resource Deletion

1. A change set shows what will be created, updated or deleted before applying changes.
2. Use change sets to avoid unexpected deletions.
3. Understand the DeletionPolicy for important resources (like S3 buckets).
4. Know that deleting a stack may delete all resources unless a retention policy is set.

Key Concepts

- ✓ **Change Set:** Preview changes before deployment.
- ✓ **DeletionPolicy:** Retain or delete resources.
- ✓ **UpdateReplacePolicy:** Controls what happens during updates.
- ✓ **Stack Deletion:** Removes resources created by the stack unless Retain is configured.

5. Delete the Stack and Verify Remaining Resources

1. If deployment failed, delete the stack to clean up.
2. Check that all resources were removed.
3. Look for any remaining resources in other services.
4. If resources remain, delete them manually.

Delete the stack

```
aws cloudformation delete-stack \
 --stack-name veriqta-s3-stack
```

Check stack status

```
aws cloudformation describe-stacks \
 --stack-name veriqta-s3-stack \
 --query "Stacks[0].StackStatus" \
 --output text
```

Verify remaining resources

1. Check the S3 console for any buckets.
2. Check IAM for any roles or policies.
3. Check other services (VPC, Security Groups, etc.).

Example: No remaining S3 buckets



Project 08. Deploy a Containerized Application

Amazon ECR, Amazon ECS, AWS Fargate

Run Your First Container on AWS

1. Build a Simple Docker Application

1. Create a simple web application (Python Flask).
2. Write a Dockerfile.
3. Build the Docker image locally.
4. Test the container on your machine.

app.py (simple Flask app)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
  return "Hello from VERIQTA
    on AWS!"

if __name__ == "__main__":
  app.run(host="0.0.0.0",
  port=5000)
```

Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY app.py .
RUN pip install flask
EXPOSE 5000
CMD ["python", "app.py"]
```

Build and test locally

```
docker build -t veriqta-app .
docker run -p 5000:5000 veriqta-app
curl http://localhost:5000
```

2. Push the Image to Amazon ECR

1. Create an ECR repository.
2. Authenticate Docker to ECR.
3. Tag your image.
4. Push the image to ECR.



AWS CLI Commands

```
aws ecr create-repository --repository-name veriqta-app
aws ecr get-login-password --region us-east-1 | \
 docker login --username AWS --password-stdin \
 123456789012.dkr.ecr.us-east-1.amazonaws.com
docker tag veriqta-app:latest 123456789012.dkr.ecr.us-
east-1.amazonaws.com/veriqta-app:latest
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/veriqta-app:latest
```

3. Create an ECS Task Definition

1. Define the container image from ECR.
2. Set CPU, memory and port mapping.
3. Attach a task role (IAM role).
4. Save and register the task definition.

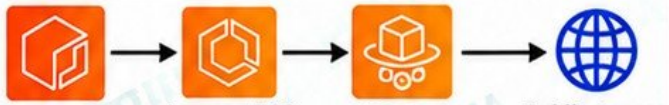
Task Definition (JSON Example)

```
{
  "family": "veriqta-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::123456789012:role/veriqta-app-role",
  "containerDefinitions": [
    {
      "name": "veriqta-app",
      "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/veriqta-app:latest",
      "portMappings": [{"containerPort": 5000, "protocol": "tcp"}]
    }
  ]
}
```

4. Run a Fargate Task

1. Create a cluster (or use an existing one).
2. Run a Fargate task using the task definition.
3. Check the task status and logs.
4. Access the application through a public IP or load balancer.

Architecture Overview



Run a Fargate Task (AWS CLI)

```
aws ecs create-cluster --cluster-name veriqta-cluster
aws ecs run-task \
  --cluster veriqta-cluster \
  --task-definition veriqta-app \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-
12345,subnet-67890],securityGroups=[sg-12345],assignPublicIp=ENABLED}"
```

5. Understand Task Roles and Networking

1. Task role gives permissions to access AWS services (e.g. S3, DynamoDB).
2. Execution role pulls the image from ECR and writes logs to CloudWatch.
3. Fargate runs tasks in your VPC using awsvpc networking.
4. Use a security group to allow inbound traffic (e.g. port 5000 or via ALB).

Key Points

- ECR stores your container image securely.
- ECS runs and manages your container tasks.
- Fargate runs containers without managing servers.
- Task role is for your application permissions.
- Execution role is used by ECS/Fargate to pull images and write logs.
- Use a load balancer for production access instead of a public IP.

Project 08. Deploy a Containerized Application

Amazon ECR, Amazon ECS, AWS Fargate

Investigate a Container That Will Not Start

1. Check ECS Task Status and Stopped Reasons

1. Open the ECS console and go to your cluster.
2. Check the task status (PENDING, RUNNING, STOPPED).
3. Click the stopped task to view the stopped reason.
4. Identify the exact error message.

ECS Tasks (Example)

Task	Last status	Desired status	Stopped reason
veriqta-app-1	✗ STOPPED	RUNNING	Essential container exited
veriqta-app-2	✓ RUNNING	RUNNING	-
veriqta-app-3	✗ STOPPED	RUNNING	CannotPullContainerError: image not found

2. Check Container Logs

1. Open Amazon CloudWatch and go to Logs.
2. Find the log group for your ECS task or container.
3. View recent log events.
4. Look for application errors, startup failures or missing files.

View logs using AWS CLI

```
aws logs describe-log-groups --log-group-name-prefix /ecs/
aws logs get-log-events \
  --log-group-name /ecs/veriqta-app \
  --log-stream-name ecs/your-task-id \
  --limit 50
```

Example application error (CloudWatch Logs)

```
2026-10-20T12:01:10.123Z ERROR [main]
FileNotFoundError: [Errno 2] No such file or directory:
'/app/config.yaml'
2026-10-20T12:01:10.124Z INFO Shutting down...
```

3. Diagnose Common Image Pull and Configuration Failures

1. Check that the ECR image URI is correct.
2. Verify the image exists and the tag is available.
3. Ensure the task execution role has ECR permissions.
4. Check container port, command and environment variables.
5. Look for configuration issues (wrong path, missing file, invalid command).

Common stopped reasons

- ✗ **CannotPullContainerError** — Image not found or no permission to pull.
- ✗ **AccessDeniedException** — Missing ECR permissions.
- ✗ **Essential container exited** — Application crashed or wrong command.
- ✗ **ResourceInitializationError** — Network, DNS or IAM issue.
- ✗ **InvalidParameterException** — Invalid DNS or IAM or container settings.

Fix examples

- ✓ Verify ECR repository and image tag.
- ✓ Attach AmazonECSTaskExecutionRole or custom policy with ECR permissions.
- ✓ Check container logs for the exact error.
- ✓ Confirm port mapping and health check.
- ✓ Update command, environment variables or file paths.

4. Verify a Successful Task Execution

1. Run a new task with the correct configuration.
2. Check that the task status becomes RUNNING.
3. Test the application (use public IP or load balancer).
4. Confirm the container is healthy and responding.

Successful task example

Task	Last status	Health status	Started at
veriqta-app-4	✓ RUNNING	✓ HEALTHY	2026-10-20 12:10:15 (UTC)

Test the application (example)

```
curl http://<public-ip>:5000/
{
  "message": "Hello from VERIQTA",
  "status": "success"
}
```

5. Stop Tasks and Delete Billable Resources

1. Tesk any running tasks when you are done testing.
2. Delete unused task defintiions (optional).
3. Delete the ECS servicep or cluster if not needed.
4. Remove the ECR repository if frayls no longer need the image.
5. Verify that no resources are left to avoid extra charges.

Stop a running task (AWS CLI)

```
aws ecs stop-task \
  --cluster veriqta-cluster \
  --task <task-id> \
  --reason "Testing completed"
```

Delete resources (AWS CLI)

```
aws ecs delete-service --cluster veriqta-cluster \
  --service veriqta-service --force
aws ecs delete-cluster --cluster veriqta-cluster
aws ecr delete-repository \
  --repository-name veriqta-app --force
```

Key Points

- ✓ Always check the stopped reason for the exact error.
- ✓ Use CloudWatch Logs to find application errors.
- ✓ Most image pull issues are caused by wrong image URI or missing ECR permissions.
- ✓ Verify configuration: command, port, environment variables and file paths.
- ✓ Confirm the task runs successfully before closing.
- ✓ Stop tasks and delete unused resources to avoid charges.

Project 09. Build an Automated Deployment Pipeline

GitHub Actions, AWS IAM OIDC, Amazon S3, CloudFront

Deploy Website Changes Automatically

VERIQTA

1. Store the Project 02 Website in GitHub

1. Create a new repository or use your existing one.
2. Push your Project 02 website files to GitHub.
3. Keep a clean folder structure (e.g. index.html, css, js, images).
4. Use a separate branch for changes (e.g. main).

Example project structure

```
veriqta-website/
├── index.html
├── css/
│   └── style.css
├── js/
│   └── app.js
└── images/
    └── logo.png
```

GitHub

2. Create a GitHub Actions Workflow

1. Add a workflow file in .github/workflows/.
2. Configure it to run on push to the main branch.
3. Install required tools (e.g. AWS CLI).
4. Upload the website files to Amazon S3.

Example workflow (deploy.yml)

```yaml
name: Deploy Website to S3
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.AWS_ROLE_ARN }}
          aws-region: us-east-1
      - name: Sync files to S3
        run: aws s3 sync ./ s3://veriqta-website --delete
```

GitHub Actions

3. Configure OpenID Connect (OIDC) Authentication

1. Create an IAM role for GitHub Actions.
2. Set up a trust policy with GitHub OIDC.
3. Allow only the required permissions (e.g. S3 access).
4. Use the role ARN in your workflow file.

IAM Trust Policy (OIDC Example)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:oidc-
-provider/token.actions.githubusccrentent.com",
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusccrentent.com:aud":
"sts.amazonaws.com",
          "token.actions.githubusccrentent.com:sub":
"repo:veriqta/veriqta-website:ref:refs/heads/main"
        }
      }
    }
  ]
}
```

IAM Permissions (Example)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::veriqta-website",
        "arn:aws:s3:::veriqta-website/*"
      ]
    }
  ]
}
```

4. Run Checks Before Deployment

1. Validate the build (e.g. check for broken links or missing files).
2. Optionally run a simple test or lint step.
3. Fail the workflow if checks do not pass.
4. Proceed to upload to S3 only when checks are successful.

Example check steps

```yaml
- name: Check for required files
  run: |
    if [ ! -f index.html ]; then
      echo "index.html not found!"
      exit 1
    fi
- name: List files (for debugging)
  run: ls -la
```

5. Upload Approved Website Changes to S3

1. Use the AWS CLI to sync your website files to the S3 bucket.
2. Set the correct bucket name and region.
3. Use --delete to remove old files that no longer exist.
4. Verify the files in the S3 console after a successful run.

S3 sync command (from workflow)

```
aws s3 sync ./ s3://veriqta-website --delete \
  --region us-east-1
```

Deployment Flow Overview



Key Points

- Use OIDC instead of long-term AWS access keys.
- Keep IAM permissions minimal (only what is needed).
- Run checks before deployment to avoid uploading broken files.
- Use S3 versioning (optional) to recover previous versions.
- You can add CloudFront later to serve your website globally.

Project 09. Build an Automated Deployment Pipeline

GitHub Actions, AWS IAM OIDC, Amazon S3, CloudFront

Recover From a Failed Deployment

VERIQTA

1. Inspect Failed Workflow Logs

1. Open the GitHub repository and go to Actions.
2. Find the failed workflow run.
3. Click the failed job to view detailed logs.
4. Identify the exact error message.

Example failed workflow log

```
Run aws s3 sync ./ s3://veriqta-website --delete
Error: AccessDenied: Access Denied
status code: 403
User: arn:aws:sts::123456789012:assumed-role/
GitHubActionsRole is not authorized to perform:
s3:PutObject on resource:
arn:aws:s3:::veriqta-website/*
```

2. Diagnose Missing Permissions and Invalid Paths

1. Check IAM role permissions (S3, CloudFront if used).
2. Verify the bucket name, region and file path.
3. Ensure the OIDC role is correctly configured.
4. Fix the issue and re-run the workflow.

Common causes

- ✗ Missing S3 permissions (s3:PutObject, s3:DeleteObject, s3:ListBucket).
- ✗ Wrong bucket name or region.
- ✗ Incorrect file path in the workflow.
- ✗ OIDC role trust policy misconfigured.
- ✗ CloudFront invalidation permission missing (cloudfront:CreateInvalidation).

Fixed S3 permissions (example)

```
{
  "Effect": "Allow",
  "Action": [
    "s3:PutObject",
    "s3:DeleteObject",
    "s3:ListBucket"
  ],
  "Resource": [
    "arn:aws:s3:::veriqta-website",
    "arn:aws:s3:::veriqta-website/*"
  ]
}
```

3. Understand Safe CloudFront Cache Invalidation

1. Use invalidation to clear old content after a successful deploy.
2. Invalidate only the necessary paths (e.g. / or specific files).
3. Wait for the invalidation to complete.
4. Avoid frequent invalidations to reduce cost.

CloudFront invalidation (AWS CLI)

```
aws cloudfront create-invalidation \
  --distribution-id E123ABC456 \
  --paths "/*"
```

Important notes

- ✓ A small website can use "/*".
- ✓ For larger websites, invalidate only changed files.
- ✓ Invalidation usually completes in a few minutes.
- ✓ Check the invalidation status in the CloudFront console.

How it works



4. Restore a Previously Working Version

1. Go to the repository and check the commit history.
2. Find a known working commit.
3. Re-deploy that version using GitHub Actions.
4. Confirm the workflow completes successfully.

Redeploy a previous commit

```
# Option 1: Re-run a previous workflow
1. Go to Actions.
2. Find a successful run.
3. Click Re-run.

# Option 2: Checkout a specific commit
git checkout <commit-hash>
git push origin main
```

5. Verify the Live Site After Recovery

1. Open the website URL in a browser.
2. Check that the latest changes are visible.
3. Test multiple pages and assets (images, css, js).
4. Use browser DevTools (Network tab) to confirm new files are loaded.

Verify using curl

```
curl -I https://your-domain.com
# line should return 200 OK
```

Check CloudFront response headers

```
curl -I https://your-domain.com
HTTP/2 200
x-cache: Hit from cloudfront
via: 1.1 abcdef.cloudfront.net (CloudFront)
```

Deployment Recovery Flow



Key Points

- ✓ Read the full workflow logs to find the root cause.
- ✓ Most deployment failures are due to missing permissions or wrong paths.
- ✓ Use CloudFront invalidation to serve the latest content.
- ✓ You can restore a previous working version from GitHub commit history.
- ✓ Always verify the live site after recovery.
- ✓ Delete any unused resources to avoid extra charges.

Project 10. Monitor and Investigate a Cloud Incident

Amazon CloudWatch, CloudWatch Alarms, SNS, Lambda

Build Your First Monitoring and Alerting System

VERIQTA

1. Reuse the Project 05 Lambda API

1. Use the existing Lambda function from Project 05.
2. Ensure the API is working and can be invoked.
3. Note the function name, region and API endpoint.
4. We will monitor this same function for errors and performance.

Architecture (project 05 API)



2. Identify Invocation, Error, and Duration Metrics

1. Open CloudWatch and go to Metrics.
2. Select AWS/Lambda namespace.
3. Find the key metrics for your function.
4. Understand what each metric tells you.

Important Lambda metrics

Metric	What it shows	Why it matters
Invocations	Number of times the function is called.	Helps you understand usage and traffic.
Errors	Number of failed invocations.	Used for alerts when function fails.
Duration	How long the function takes to run.	Helps detect performance issues or slow responses.
Throttles	Requests rejected due to limits.	Showts when you need a higher limit or optimization.
ConcurrentExecutions	Number of concurrent invocations.	Helps you understand scaling and load.

3. Configure a CloudWatch Alarm

1. Choose the Errors metric for your Lambda function.
2. Set a threshold (e.g. greater than or equal to 1 error).
3. Choose the evaluation period (e.g. 1 out of 1 minute).
4. Select the SNS topic for notifications.
5. Give the alarm a clear name and description.

Alarm configuration (example)

Metric	Lambda > Errors
Function name	veriqta-api
Statistic	Sum
Period	1 minute
Threshold type	Static
Threshold value	Greater than or equal to 1
Evaluation periods	1 out of 1
Alarm name	veriqta-api-errors
Alarm state	In alarm, send notification to SNS

4. Create an SNS Email Notification

1. Create a new SNS topic.
2. Add your email address as a subscriber.
3. Confirm the subscription from your email.
4. Attach this SNS topic to the CloudWatch alarm.

SNS notification flow



SNS subscription (AWS CLI example)

```
aws sns create-topic --name veriqta-api-alerts
aws sns subscribe \
  --topic-arn arn:aws:sns:us-east-1:123456789012:veriqta-api-alerts \
  --protocol email \
  --notification-endpoint your-email@example.com
```

5. Understand What Monitoring Can and Cannot Detect

What monitoring can detect

- Function errors and failures.
- Sudden increase or drop in traffic.
- High duration (slow responses).
- Throttling due to limits.
- System metrics (CPU, memory, etc.).

What monitoring cannot detect

- Logical errors (wrong output).
- Business logic issues.
- Data quality problems.
- Security vulnerabilities.
- Whether the response is correct for users.

Key Points

- Monitor key metrics: Invocations, Errors, Duration.
- Use CloudWatch alarms to get notified early.
- SNS sends email notifications when an alarm is triggered.
- Monitoring tells you that something is wrong, not always the cause.
- Combine monitoring with logs and tracing for full investigation.

Project 10. Monitor and Investigate a Cloud Incident

Amazon CloudWatch, CloudWatch Alarms, SNS, Lambda

Investigate and Recover From an AWS Incident

1. Trigger a Controlled Lambda Failure

1. Modify your Lambda function to throw an error intentionally.
2. Invoke the function to generate a failure.
3. Confirm that the function invocation fails.
4. This helps you test your monitoring and alerting system.

Example: Lambda function with a controlled failure

```
import json
def lambda_handler(event, context):
    raise Exception("Intentional error for testing")
```

2. Observe Metrics and Alarm State

1. Go to CloudWatch Metrics and check the Errors metric.
2. Wait for the alarm to change to ALARM state.
3. Check the alarm history for details.
4. Confirm that you received the SNS email notification.

Alarm flow during a failure



3. Investigate CloudWatch Logs

1. Open CloudWatch and go to Logs.
2. Find the log group for your Lambda function.
3. Check the log stream for the failed invocation.
4. Read the error message and stack trace.

Example CloudWatch log (failed invocation)

```
START RequestId: 12345678-1234-1234-1234-123456789012
Error: Intentional error for testing
    at lambda_handler (index.py:4:11)
    at Runtime.handleOnce (runtime.js:66:25)
END RequestId: 12345678-1234-1234-1234-123456789012
REPORT RequestId: 12345678-1234-1234-1234-123456789012
    Duration: 15.32 ms    Billed Duration: 100 ms
    Memory Size: 128 MB    Max Memory Used: 58 MB
```

4. Identify the Failure and Restore the Function

1. Identify the root cause (code error, configuration, permissions, etc.).
2. Fix the issue in your Lambda function.
3. Deploy the updated code.
4. Invoke the function again to confirm it works.

Example: Fixed Lambda function

```
import json
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": json.dumps({"message": "Function is working"})
    }
```

5. Verify Recovery and Document Lessons Learned

1. Check that the function now runs successfully.
2. Confirm that the CloudWatch alarm returns to OK state.
3. Verify that no new error notifications are received.
4. Test the function multiple times.
5. Document what caused the issue, how you fixed it, and what you learned.

Successful invocation (after fix)

```
{
  "statusCode": 200,
  "body": "{\"message\": \"Function is working\"}"
}
```

Alarm state after recovery

Alarm state
OK
No recent breaches

6. Remove Monitoring Resources and Final Cleanup

1. Delete the CloudWatch alarm.
2. Delete the SNS topic (if no longer needed).
3. Remove any test log groups (optional).
4. Ensure there are no unnecessary resources left running.
5. This helps you avoid extra charges.

Cleanup commands (AWS CLI examples)

```
aws cloudwatch delete-alarms --alarm-names veriqta-lambda-alarm
aws sns delete-topic --topic-arn arn:aws:sns:us-east-1:123456789012:veriqta-alerts
aws logs delete-log-group --log-group-name /aws/lambda/veriqta-api
```

Key Points

- Use controlled failures to test your monitoring system.
- CloudWatch metrics and alarms help you detect issues early.
- Logs show the exact error and help you find the root cause.
- Fix the issue, verify recovery, and document what you learned.
- Always clean up monitoring resources to avoid extra costs.
- Monitoring helps detect and alert, but it does not fix problems automatically.